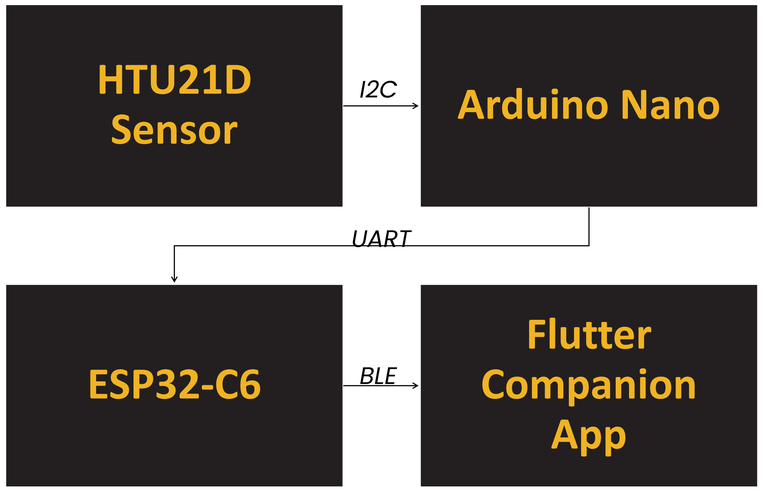
StillCold: Environmental Monitoring Without Opening the Door

Jeffrey Perdue, Dr. Samuel Cho
School of Computing and Analytics, College of Informatics, Northern Kentucky University

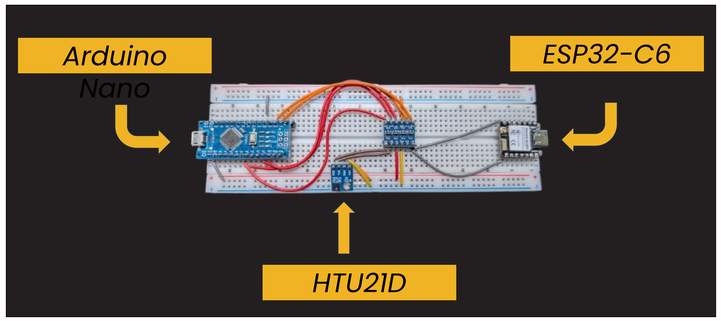
Introduction / Motivation

- Opening a refrigerator to check the temperature disrupts the environment and speeds spoilage; StillCold enables monitoring without opening the door.
- Bacteria can grow at slightly unsafe temperatures before food looks spoiled; StillCold provides continuous, early detection.
- Uses Bluetooth Low Energy (BLE) only—no Wi-Fi or cloud—keeping data local and running on a USB power bank even during outages.

System Architecture



Hardware

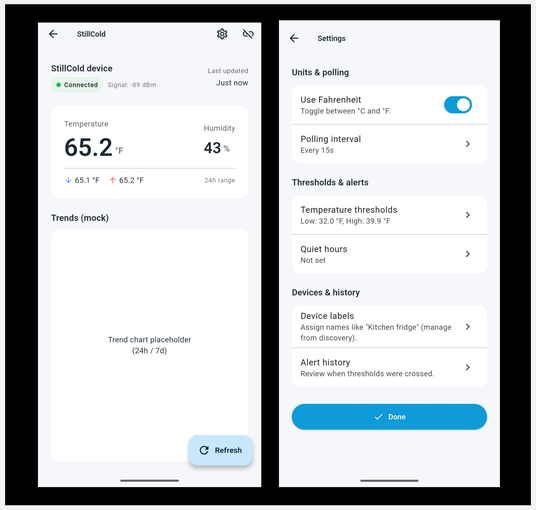


Component	Role
HTU21D Sensor	Measures temp & humidity inside the closed environment
Arduino Nano	Reads the sensor and passes the data along
ESP32-C6	Wirelessly broadcasts the latest temp & humidity to a smartphone via Bluetooth
Anker Power Bank	Powers the entire device

Flutter Companion App

- Runs on Android; discovers and connects to StillCold automatically over Bluetooth Low Energy
- Live dashboard displays current temperature and humidity — no manual refresh needed
- Readings timestamped and stored locally in SQLite (database) — no internet or cloud account required
- Customizable high/low temperature thresholds trigger alerts when conditions drift outside a safe range
- Full alert history log records when thresholds were crossed and the reading at that time

Flutter is a framework made by Google that allows you to write code once, and deploy it across multiple platforms (Android, iOS, Web)



Flutter companion app — live dashboard (left) and alert configuration (right)

How I Used AI to Solve the Problem

StillCold required learning several unfamiliar technologies simultaneously. AI served as an on-demand tutor and thought partner throughout the build.

Learning the Basics

- To learn C++ (the language powering the microcontrollers) and understand how a breadboard works — the platform used to wire and test the hardware prototype.

Building the Hardware

- To reason through wiring decisions, diagnose voltage mismatches, and evaluate component choices before testing them on the bench.

Building the App

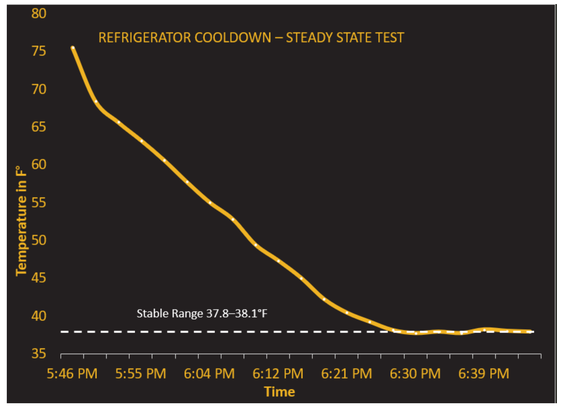
- To accelerate Flutter development, structure the codebase, and translate project requirements into working software features.

The Bigger Picture

- AI was most valuable for asking "why" before "how" — understanding tradeoffs rather than just getting answers. Every decision was ultimately verified through building and testing.

Cold Environment Testing

- Hardware sealed in a Ziploc bag with bubble wrap and placed in a household refrigerator.
- Three scenarios tested: steady-state cooldown, door open/close events, and post-power-cycle reconnection. **All three passed.**



Temperature recorded every 1–2 min via the Flutter app. Dashed line indicates stable fridge range (37.8–38.1°F).

Scenario	Key Result
Short door open (~35 sec)	+0.9°F peak, recovered in ~12 min
Prolonged Door Open	+2.6°F peak, recovered in ~23 min
Power Cycle (10 trials)	Valid reading in 10.8s avg (target: <15s)

Readings remained within ±0.25°F over 16 minutes at steady state — sufficient precision for food safety monitoring.

Next Steps

- Migrate hardware from breadboard to perfboard for a more durable assembly
- Explore single-MCU design by connecting the sensor directly to the ESP32-C6, removing the Arduino Nano
- Temperature history trend charts in the Flutter app
- iOS compatibility verification

Acknowledgements

The author thanks Dr. Samuel Cho for guidance and mentorship throughout this project. This work was supported by the School of Computing and Analytics, College of Informatics, Northern Kentucky University, and CINSAM.